Implementation and Analysis of of Graph Algorithms for TSP, CPP, and Knight's Tour

Graph Theory — Midterm Assignment



Ahmad Farras Favian Al Efasi - 5025251005

Daniel Pedrosaputra - 5025251171

Institut Teknologi Sepuluh Nopember

2026

1. PROBLEM INTRODUCTION

The Travelling Salesman Problem (TSP), Chinese Postman Problem (CPP), and Knight's Tour are three problems that involve finding an efficient or valid path under specific constraints. Although they share the concept of moving through a network of connected points, each problem has a different objective. TSP focuses on visiting every location exactly once and returning to the starting point with the minimum total cost. CPP focuses on traversing every required edge at least once while minimizing the total travel cost and returning to the starting point. In contrast, the Knight's Tour focuses on finding a sequence of legal knight moves that visits every square on a chessboard exactly once.

These problems represent different challenges in pathfinding and route planning. TSP and CPP can be applied to transportation and logistics, exactly as the MBG delivery problem, where route efficiency is important to preserve the meal. TSP can help determine an efficient delivery route between multiple destinations, while CPP can be used to plan services that must cover a network of roads, such as DLH garbage collection problems. The Knight's Tour, on the other hand, demonstrates how a sequence of valid moves can be constructed under movement restrictions and visitation requirements.

The main challenge is to develop algorithms that satisfy each problem's constraints while producing valid solutions efficiently. By implementing these three problems, this project explores different approaches to graph traversal, path optimization, and constrained pathfinding, as well as the computational complexity and limitations associated with each algorithm.

2. ALGORITHMS

a. Travelling Salesman Problem (TSP)

The Travelling Salesman Problem (TSP) finds the shortest closed tour that starts at one location and visits every other location exactly once. The number of possible tours grows factorially, so the optimal solution becomes expensive as locations increase. In this project, TSP plans mealbox delivery from an SPPG to schools in Surabaya over the actual road network.

The road network is a weighted directed graph: vertices are intersections and directed edges are road segments weighted by length, so one-way streets make the distance from A to B differ from B to A. Because a route may reuse roads, we snap the SPPG and the k selected schools to their nearest vertices, run Dijkstra from each to build a $(k + 1) \times (k + 1)$ asymmetric distance matrix D, solve the TSP on D, and expand each leg into its real shortest road.

Held-Karp ($k \leq 12$ schools):

Dynamic programming over subsets, where dp[S][j] is the minimum cost of a path from the SPPG through the schools in S that ends at school j. Time complexity is $O(2^k \cdot k^2)$.

Nearest Neighbour + 2-opt/Or-opt ($k \geq 12$ schools):

Build a tour by always visiting the closest unvisited school, then improve it with 2-opt (segment reversal) and Or-opt (moving 1 to 3 stops). Perturbation restarts run for about 6 seconds and the best tour is kept. All moves use the true directed cost.

b. **Chinese Postman Problem (CPP)**

The Chinese Postman Problem (CPP) finds the shortest closed route that starts at one location and drives along every road at least once. Unlike the TSP, which chooses which places to visit, the CPP must cover all the roads, and the route may reuse a road when that is cheaper than any other way to finish. In this project, the CPP plans the route of a dump truck over the trunk, primary, secondary and tertiary roads within x km of a base in Surabaya.

The road network is a weighted directed graph: vertices are intersections and directed edges are road segments weighted by length, so one-way streets can only be driven in one direction. We keep only the roads within x km of the base, reduce the network to its largest strongly connected component so that a closed route exists, snap the base to its nearest vertex, balance the graph, and build the route from it.

A closed route that drives every road exactly once exists only if, at every intersection, the number of roads coming in equals the number going out. Real road networks are not balanced, so some roads must be driven a second time. An intersection with more roads going out than coming in needs extra arrivals, and an intersection with more roads coming in than going out must send out extra traversals. We model this as a minimum-cost flow, where the supply and demand at each vertex are the difference between its in-degree and out-degree, and the cost of an edge is its length. The flow shows the cheapest set of roads to repeat, and these roads are added to the graph as extra edges. When roads run in parallel between the same two intersections, the shortest one is used for repetition.

After balancing, every vertex has equal in-degree and out-degree, so an Euler circuit exists. We build it with Hierholzer's algorithm, starting and ending at the base. The order of the edges in the circuit is the street traversal order, and the total distance is the sum of the edge lengths along it.

Checking strong connectivity takes $O(V + E)$, and building the Euler circuit takes $O(E)$, where V is the number of intersections and E is the number of road segments (including the repeated ones). The balancing step is the most expensive part, because it solves a minimum-cost flow with the network simplex method. Unlike the TSP, the directed CPP is solvable in polynomial time, so the solver stays fast on real road networks. In our test with 3,063 road segments, it took 358 ms.

c. Knight's Tour Problem

The Knight's Tour Problem asks for a sequence of knight moves on a chessboard such that the knight visits every square exactly once. The number of possible move sequences grows exponentially with the size of the board, so the search must avoid exploring dead ends. In this project, the Knight's Tour is generated on a dynamic chessboard with a given number of columns and rows and a given starting square.

The chessboard is modeled as an undirected graph: vertices are squares, and an edge connects two squares if a knight can jump between them in one L-shaped move (two squares in one direction and one square perpendicular). A Knight's Tour is therefore a Hamiltonian path in this graph, starting from the given square. Because not every board and starting square admits a tour (for example, a 3 × 3 board, or a 5 × 5 board started from a square of the opposite color to the corners), the solver prints "path not found" when the search is exhausted without success.

Backtracking:

Depth-first search that extends the tour one move at a time. From the current square, the knight moves to an unvisited square reachable by a legal knight move. If no such square exists before all N squares are visited, the last move is undone (backtracked) and the next alternative is tried. The search ends when all N = columns × rows squares are visited, or when every alternative from the starting square has been tried. Worst-case time complexity is O(8^N).

Warnsdorff's Rule:

A heuristic that decides the order in which backtracking tries the moves. From the current square, the unvisited neighbors are sorted by their degree, the number of unvisited squares each one can still reach, and the neighbor with the fewest onward moves is tried first. This visits hard-to-reach squares such as corners before they become isolated. Computing the degrees takes constant time per move (at most 8 neighbors, each with at most 8 moves), so when the heuristic finds a tour without backtracking the time complexity is O(N). Backtracking is kept as a fallback so that the solver is still complete: if the heuristic order leads to a dead end, the search recovers and either finds another tour or correctly reports that none exists.

3. PROBLEM IMPLEMENTATION

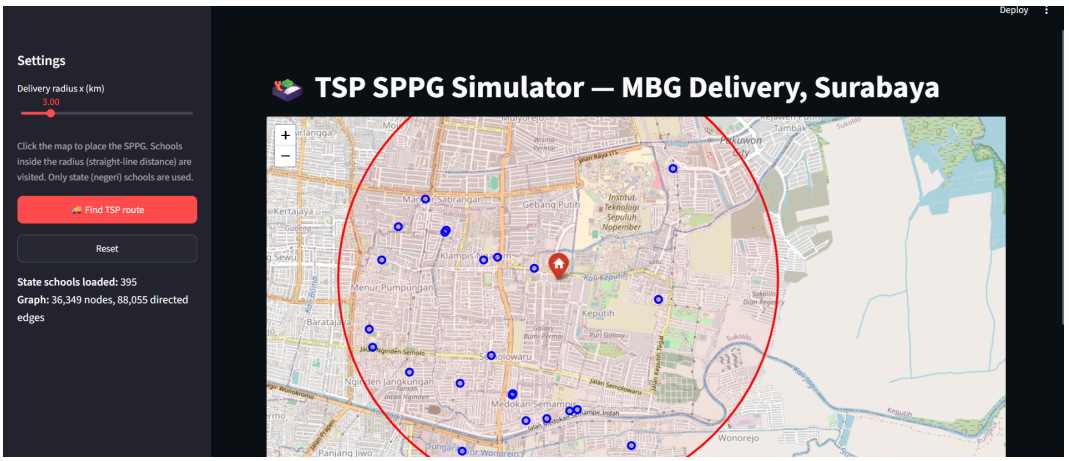
a. SPPG Simulator

The SPPG Simulator is written in Python. Road and school data come from OpenStreetMap (OSM) instead of manual Google Maps measurements, so any location in Surabaya can be selected.

- OSMnx downloads the drivable road network of Surabaya and the schools (OSM tag amenity=school, using the centroid of each school).
- The network is reduced to its largest strongly connected component so that every vertex can reach every other vertex, then converted to a NetworkX DiGraph that respects one-way streets.
- Streamlit and Folium provide the interactive web map. Data is cached locally after the first download.

Workflow**:**

1. The user clicks the map to place the SPPG and moves a slider to set the radius x km.
2. Schools whose straight-line (haversine) distance from the SPPG is at most x km are selected.
3. The SPPG and schools are snapped to the nearest road vertices.
4. Dijkstra builds the directed distance matrix.
5. The solver (Held-Karp or the heuristic) finds the visiting order, and its execution time is measured.
6. Each leg is expanded into the real road path using the shortest-path result, and the route is drawn on the map.
7. Total distance is the sum of the edge lengths along the route. Estimated travel time is the sum of OSM edge travel times, computed from speed limits (or OSM default speeds by road type), without live traffic.



map with the selected SPPG, delivery radius and school markers

b. The Dump Truck Route Simulator

The Dump Truck Route Simulator is written in Python. Road data come from OpenStreetMap (OSM) instead of manual Google Maps measurements, so any location in Surabaya can be selected as the dump truck base.

- OSMnx downloads the drivable road network around the selected base. Only trunk, primary, secondary and tertiary roads (and their link roads) are requested.
- The network is reduced to its largest strongly connected component, so that a closed route exists and unconnected road segments are ignored, then kept as a NetworkX MultiDiGraph that respects one-way streets.
- Streamlit and Folium provide the interactive web map. Data is cached locally after the first download.

Workflow:

1. The user clicks the map to place the dump truck base and moves a slider to set the radius x km.
2. Road segments whose two ends are within x km (straight-line, haversine distance) of the base are selected.
3. The base is snapped to the nearest road vertex.

4. The solver balances the network, so that at every intersection the roads coming in equal the roads going out. A minimum-cost flow finds the cheapest roads to drive a second time, and its execution time is measured.
5. An Euler circuit (Hierholzer's algorithm) is built on the balanced graph, starting and ending at the base, and the route is drawn on the map.
6. Total distance is the sum of the edge lengths along the route. Estimated travel time is the total distance divided by an assumed constant truck speed (30 km/h by default, adjustable),



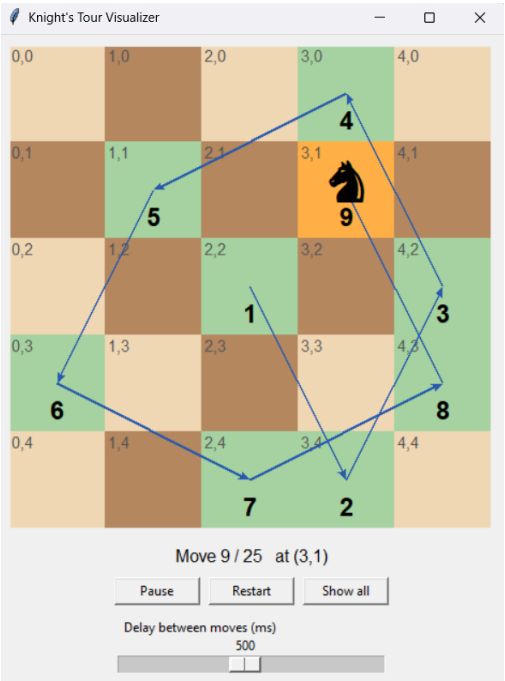
c. Knight's Tour Simulator

The Knight's Tour Simulator is written in Python. Unlike the SPPG and dump truck simulators, it needs no map data, because the chessboard is generated from the input values, so any board size can be used.

- The chessboard is built from the number of columns and rows given by the user. Each square is identified by its coordinates (column, row), with (0, 0) as the top-left square.
- The solver combines backtracking with Warnsdorff's rule. It finds the tour by trying legal knight moves in order of fewest onward moves and undoing a move whenever it reaches a dead end.
- Tkinter provides the animated visualization of the finished tour. It reuses the same solver, so the console output and the animation always show the same tour.

Workflow:

1. The user enters the number of columns and rows of the chessboard, for example 5 5.
2. The user enters the starting position of the knight (column and row), for example 2 2.
3. The solver checks that the starting square lies inside the board. If it does not, the program prints “path not found”.
4. From the current square, the solver finds all unvisited squares reachable by a knight move and sorts them by the number of onward moves each one has (Warnsdorff’s rule).
5. The solver moves to the first square in that order. If a square has no unvisited moves left before all squares are visited, the last move is undone and the next option is tried (backtracking).
6. When all squares have been visited exactly once, the coordinates are printed in visiting order, one per line, written as column,row. If every option is exhausted, the program prints “path not found”.
7. The Tkinter window animates the tour on the chessboard. The knight moves square by square, the path is drawn as arrows, each square is marked with its move number, and the current square is highlighted. The animation can be paused, restarted, shown in full at once, and sped up or slowed down.



chessboard with the starting position, the numbered visit order, and the knight’s path

4. PROBLEM OUTPUT

a. SPPG Simulator

The simulator displays the suggested route overlaid on the map, the school visiting order, the total distance and estimated travel time, and the solver execution time.

Test configuration:

1. SPPG location: [name or coordinates]
2. delivery radius: [x] km
3. number of schools in radius: [k]
4. solver used: [Held-Karp / heuristic].



Order	Stop	Leg distance (km)	Leg time (min)
0	SPPG (start)	None	None
1	SD. Al Falah	0.84	1.2
2	SMP Al Falah Surabaya	0.26	0.6
3	SD Negeri Keputran VI No. 337	0.89	1.3
4	SD Negeri Dr. Soetomo V-VIII	1.11	2.4
5	TK-SD Pakis Gelora	0.6	1.6
6	SD Negeri Pakis I-II & IX	1.14	5.4
7	SD-SMP Rahmat	1.15	6.4
8	Lembaga Ilmu Pengetahuan Islam and Arab cabang Surat	0.65	0.9
9	SD Kristen Petra 1	1.05	1.6

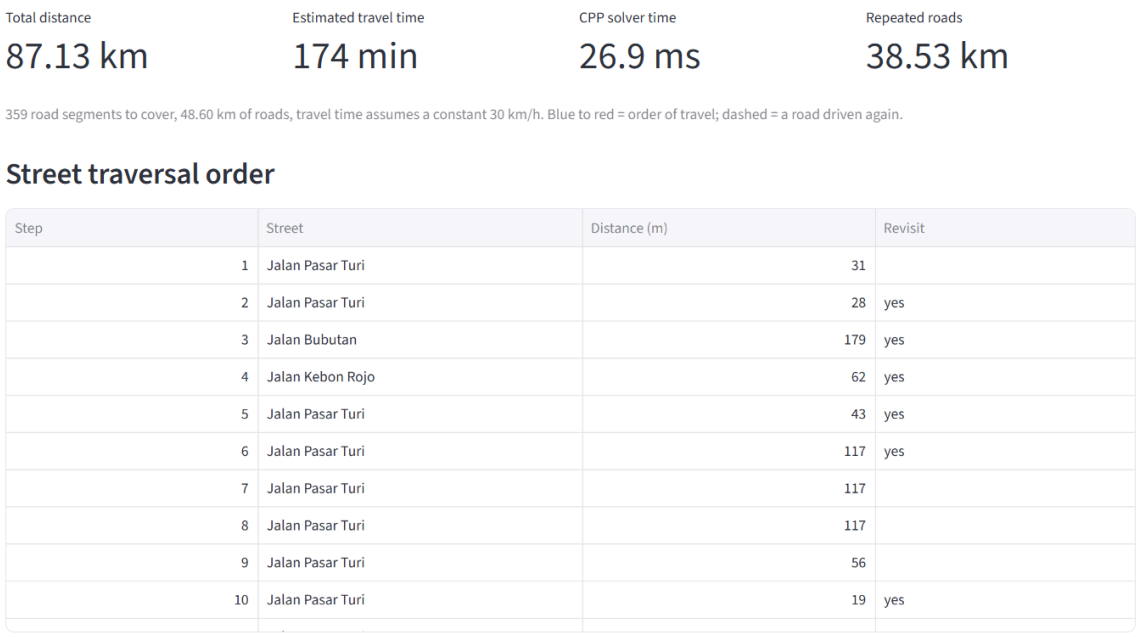
route overlaid on the map with numbered stops

b. Dump Truck Route Simulator

The simulator displays the suggested route overlaid on the map, the street traversal order, the total distance and estimated travel time, and the solver execution time.

Test configuration:

1. Dump truck base location: (-7.241065, 112.644806)
2. Delivery radius: 6 km
3. Number of road segments in radius: 566
4. Solver used: directed CPP (minimum-cost flow balancing and Euler circuit)



Total distance	Estimated travel time	CPP solver time	Repeated roads
87.13 km	174 min	26.9 ms	38.53 km

359 road segments to cover, 48.60 km of roads, travel time assumes a constant 30 km/h. Blue to red = order of travel; dashed = a road driven again.

Street traversal order

Step	Street	Distance (m)	Revisit
1	Jalan Pasar Turi	31	
2	Jalan Pasar Turi	28	yes
3	Jalan Bubutan	179	yes
4	Jalan Kebon Rojo	62	yes
5	Jalan Pasar Turi	43	yes
6	Jalan Pasar Turi	117	yes
7	Jalan Pasar Turi	117	
8	Jalan Pasar Turi	117	
9	Jalan Pasar Turi	56	
10	Jalan Pasar Turi	19	yes

summary of the CPP result, showing the total distance (87.13 km), estimated travel time (174 min), solver execution time (26.9 ms), and repeated road distance (38.53 km).

c. Knight's Tour Simulator

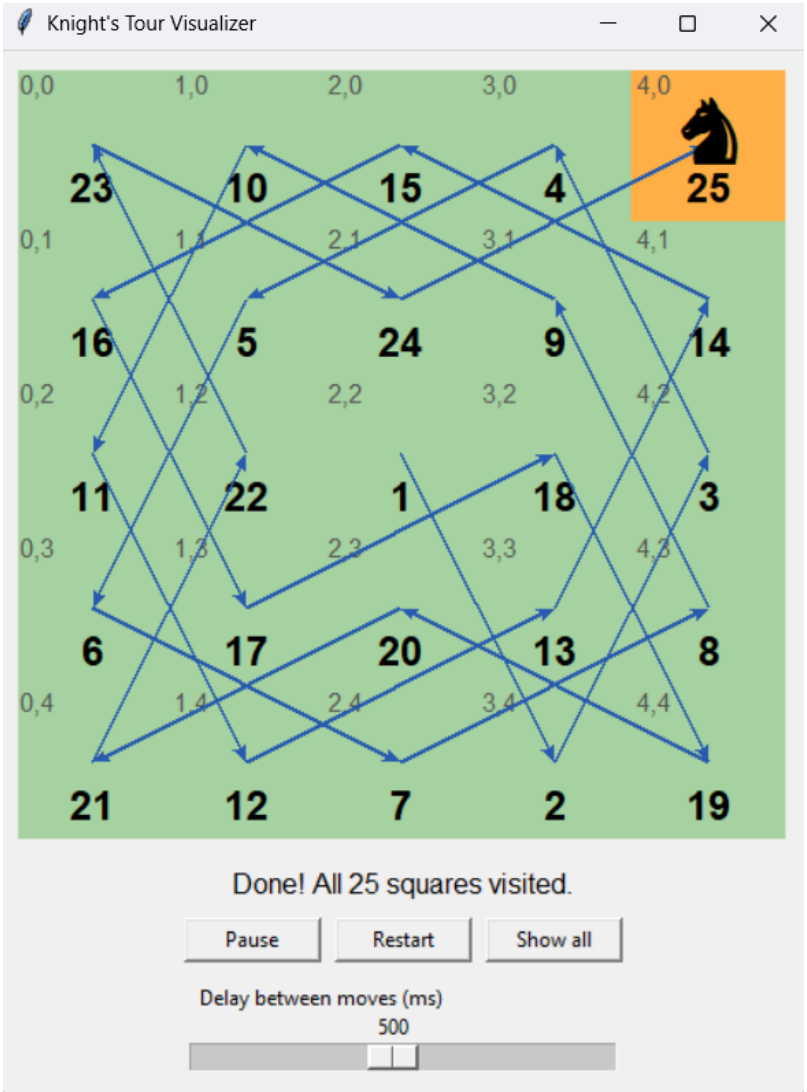
The simulator displays the knight's complete tour on the chessboard, including the visiting order of every square, the path of the knight, and the list of coordinates visited. If no tour exists for the given board and starting position, it prints "path not found".

Test configuration:

1. Chessboard size: 5 × 5 (25 squares)
2. Starting position: (2, 2)
3. Solver used: backtracking with Warnsdorff's rule
4. Result: tour found, 25 of 25 squares visited in 24 moves, ending at (4, 0)

[insert console output screenshot here]

console output showing the coordinates (column, row) visited by the knight in order



5 × 5 chessboard showing the knight's completed tour from (2, 2), with the numbered visit order and the path drawn as arrows.

6. CONCLUSION

The three problems were implemented and tested in Python. Each one ended up as a separate simulator.

The Travelling Salesman Problem was applied to MBG mealbox delivery over the Surabaya road network. Held-Karp finds the optimal tour with dynamic programming, but its O(2^k · k^2) complexity limits it to small numbers of schools. For larger instances, Nearest Neighbour with 2-opt and Or-opt gives a good tour within a few seconds, though it does not guarantee the optimum. In the SPPG test, the heuristic visited 33 schools over 21.71 km with an estimated travel time of 55 minutes.

The Chinese Postman Problem was applied to dump truck route planning. Balancing the directed graph with minimum-cost flow and then building an Euler circuit with Hierholzer's algorithm gives a closed route that covers every required road. The balancing step is the most expensive part, but the directed CPP is solvable in polynomial time, so the solver stayed fast: 26.9 ms in the 6 km test. The route covered 87.13 km, of which 38.53 km were repeated roads.

The Knight's Tour was solved on a dynamic chessboard using backtracking guided by Warnsdorff's rule. Backtracking has a worst-case complexity of O(8^N), but Warnsdorff's rule tries the squares with the fewest onward moves first, so a tour is usually found with little or no backtracking. Backtracking is kept as a fallback, so the solver is still complete and prints "path not found" when no tour exists. On a 5 × 5 board starting from (2, 2), it visited all 25 squares in 24 moves.

Overall, the three problems show different sides of graph traversal. The TSP is NP-hard, so large instances need heuristics. The directed CPP can be solved efficiently and exactly. The Knight's Tour is a Hamiltonian path problem, where a good heuristic makes the search practical. In all three, the choice of algorithm depends on the problem size and on whether an exact or an approximate answer is acceptable.

AI Tools and Prompts

Tools Used: ChatGPT and Claude

The following prompts represent the types of questions used to support the development and documentation of the project:

1. **Understanding the Project Requirements**
   "Help me break down the assignment requirements and identify what needs to be implemented for the Travelling Salesman Problem, Chinese Postman Problem, and Knight's Tour. Explain the expected inputs, outputs, and evaluation criteria for each algorithm."
2. **Selecting and Understanding Algorithms**
   "Compare possible approaches for solving TSP, CPP, and Knight's Tour. Explain the advantages, limitations, and computational complexity of each approach so I can understand why the selected algorithms are suitable for our project."
3. **Developing the TSP Implementation**
   "Help me design the TSP implementation using OpenStreetMap road data. Explain how to calculate distances between locations, account for one-way roads, and compare Held-Karp with Nearest Neighbour combined with local optimization."
4. **Implementing and Testing the CPP Algorithm**
   "Guide me through implementing the Chinese Postman Problem for a road network. Explain how to balance the graph, minimize repeated travel distance, construct an Eulerian circuit, and verify the resulting route."
5. **Improving the Knight's Tour Solver**
   "Help me understand how backtracking and Warnsdorff's rule can be combined to solve Knight's Tour. Explain how to validate legal knight moves, track visited squares, and determine whether all squares have been visited."
6. **Evaluating Algorithm Performance**
   "Help me evaluate the output of each implementation. Identify which measurements are relevant, including total route distance, estimated travel time, repeated road distance, execution time, and the number of squares visited. Explain how to interpret the results."
7. **Checking Correctness and Handling Errors**
   "Review the algorithm's logic and help me identify possible edge cases or errors.

Suggest test cases that can verify whether the TSP route, CPP circuit, and Knight's Tour satisfy their respective requirements."

8. **Preparing Output Visualizations**

   "Help me improve the presentation of the algorithm outputs using maps and route visualizations for TSP and CPP, along with a numbered board visualization for Knight's Tour. Explain what information should be visible to make the results easy to understand."

9. **Documenting the Implementation**

   "Help me organize the implementation details, algorithm explanations, testing results, and screenshots into a clear technical report. Suggest suitable figure captions and explain how the results should be connected to the objectives of the project."

10. **Reviewing the Final Report**

    "Review the report for clarity, consistency, and technical accuracy. Check whether the explanation of each algorithm matches its implementation and output, and help improve the conclusion without introducing results that were not obtained from testing."